

Understanding Reinforcement Learning from a Mathematical View

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

October 7, 2025

The theory of dynamic programming is the mathematical foundation of sequential decision making.

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

What is Reinforcement Learning?

Reinforcement Learning (RL) is a machine learning paradigm where an agent learns to make decisions by interacting with an environment.

- **Agent**: The decision maker
- **Environment**: The world the agent interacts with
- **State**: Current situation of the environment
- **Action**: What the agent can do
- **Reward**: Feedback from the environment

The goal is to learn a **policy** that maximizes cumulative reward.

Mathematical Framework: Markov Decision Process

A Markov Decision Process (MDP) is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$: State space
- $\mathcal{A}$: Action space
- $\mathcal{P}(s'|s, a)$: Transition probability
- $\mathcal{R}(s, a)$: Reward function
- $\gamma \in [0, 1]$: Discount factor

Markov Property: Future depends only on current state

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t) \quad (1)$$

Policy and Value Functions

A **policy** $\pi(a|s)$ defines the probability of taking action a in state s .

Value Functions:

- State Value Function:

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right] \quad (2)$$

- Action Value Function:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s, a_0 = a \right] \quad (3)$$

Optimal Value Functions:

$$V^*(s) = \max_{\pi} V^{\pi}(s), \quad Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a) \quad (4)$$

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations**
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

Bellman Equation for State Value

The state value function satisfies the **Bellman equation**:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a) + \gamma V^\pi(s')] \quad (5)$$

Optimal Bellman Equation:

$$V^*(s) = \max_a \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a) + \gamma V^*(s')] \quad (6)$$

Action Value Bellman Equation:

$$Q^\pi(s, a) = \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a) + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s', a')] \quad (7)$$

Bellman Optimality for Action Values

Optimal Action Value Bellman Equation:

$$Q^*(s, a) = \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a) + \gamma \max_{a'} Q^*(s', a')] \quad (8)$$

Optimal Policy:

$$\pi^*(s) = \arg \max_a Q^*(s, a) \quad (9)$$

Key Insight: If we know Q^* , we can find the optimal policy by taking the action with highest value in each state.

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming**
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

Policy Evaluation

Given a policy π , how to compute V^π ?

Iterative Policy Evaluation:

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a) + \gamma V_k(s')] \quad (10)$$

Convergence: $V_k \rightarrow V^\pi$ as $k \rightarrow \infty$

Matrix Form:

$$\mathbf{V}_{k+1} = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \mathbf{V}_k \quad (11)$$

where $\mathbf{P}^\pi$ is the transition matrix under policy π .

Policy Improvement

Given V^π , how to find a better policy?

Policy Improvement Theorem: For any policy π , define a new policy π' :

$$\pi'(s) = \arg \max_a \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a) + \gamma V^\pi(s')] \quad (12)$$

Then $V^{\pi'}(s) \geq V^\pi(s)$ for all s .

Proof Sketch:

$$V^\pi(s) \leq \max_a \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a) + \gamma V^\pi(s')] = V^{\pi'}(s) \quad (13)$$

Policy Iteration Algorithm:

- ① Initialize policy π_0 randomly
- ② **Policy Evaluation:** Compute V^{π_k}
- ③ **Policy Improvement:**

$$\pi_{k+1}(s) = \arg \max_a \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a) + \gamma V^{\pi_k}(s')] \quad (14)$$

- ④ Repeat until $\pi_{k+1} = \pi_k$

Convergence: Guaranteed to find optimal policy in finite steps.

Value Iteration Algorithm:

$$V_{k+1}(s) = \max_a \sum_{s'} \mathcal{P}(s'|s, a) [\mathcal{R}(s, a) + \gamma V_k(s')] \quad (15)$$

Convergence: $V_k \rightarrow V^*$ as $k \rightarrow \infty$

Advantage: No need for separate policy evaluation step.

Stopping Criterion: $\max_s |V_{k+1}(s) - V_k(s)| < \epsilon$

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods**
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

Monte Carlo Policy Evaluation

When transition probabilities $\mathcal{P}$ are unknown, use **Monte Carlo** methods.

First-Visit MC:

$$V(s) \leftarrow V(s) + \alpha[G_t - V(s)] \quad (16)$$

where $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$ is the return.

Every-Visit MC: Update $V(s)$ every time s is visited in an episode.

Convergence: $V(s) \rightarrow V^\pi(s)$ as number of visits $\rightarrow \infty$

MC Control Algorithm:

- ① Initialize $Q(s, a)$ and $\pi(s)$ arbitrarily
- ② For each episode:
 - Generate episode following π
 - For each (s, a) pair in episode:
 - $Q(s, a) \leftarrow Q(s, a) + \alpha[G_t - Q(s, a)]$
 - $\pi(s) \leftarrow \arg \max_a Q(s, a)$

Exploration Problem: Need to ensure all actions are tried.

ϵ -Greedy Policy

ϵ -Greedy Policy:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{if } a = \arg \max_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise} \end{cases} \quad (17)$$

GLIE (Greedy in the Limit with Infinite Exploration):

- All state-action pairs visited infinitely often
- Policy converges to greedy policy

Example: $\epsilon_k = \frac{1}{k}$ where k is episode number.

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning**
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

Temporal Difference (TD) Learning

TD(0) Algorithm:

$$V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)] \quad (18)$$

TD Error:

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t) \quad (19)$$

Advantages over MC:

- Online learning (no need to wait for episode end)
- Lower variance than MC
- Can learn from incomplete sequences

SARSA: On-Policy TD Control

SARSA Algorithm:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (20)$$

On-Policy: Learns the value of the policy being followed.

Algorithm:

- 1 Choose a_t from s_t using policy derived from Q
- 2 Take action a_t , observe r_{t+1}, s_{t+1}
- 3 Choose a_{t+1} from s_{t+1} using policy derived from Q
- 4 Update $Q(s_t, a_t)$

Q-Learning: Off-Policy TD Control

Q-Learning Algorithm:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (21)$$

Off-Policy: Learns optimal policy while following different behavior policy.

Convergence: $Q(s, a) \rightarrow Q^*(s, a)$ under certain conditions:

- All state-action pairs visited infinitely often
- Learning rate satisfies Robbins-Monro conditions

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation**
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

Linear Function Approximation

When state space is large, use function approximation:

$$\hat{V}(s) = \mathbf{w}^T \phi(s) \quad (22)$$

where $\phi(s)$ are features and $\mathbf{w}$ are weights.

Linear TD(0):

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha[r_{t+1} + \gamma \mathbf{w}_t^T \phi(s_{t+1}) - \mathbf{w}_t^T \phi(s_t)] \phi(s_t) \quad (23)$$

Convergence: Guaranteed to converge to $\mathbf{w}^*$ that minimizes mean squared error.

Neural Network Function Approximation

Deep Q-Network (DQN):

$$Q(s, a; \theta) \approx Q^*(s, a) \quad (24)$$

Loss Function:

$$L(\theta) = \mathbb{E}[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2] \quad (25)$$

Key Innovations:

- **Experience Replay**: Store transitions and sample randomly
- **Target Network**: Use separate network for target values
- **Double DQN**: Reduce overestimation bias

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods**
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

Policy Gradient Theorem

Objective: Maximize expected return

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (26)$$

Policy Gradient Theorem:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t}^T \gamma^{t'-t} r_{t'} \right] \quad (27)$$

REINFORCE Algorithm:

$$\theta_{t+1} = \theta_t + \alpha \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \quad (28)$$

Actor-Critic Methods

Actor-Critic: Combine policy gradient with value function approximation.

Actor Update (Policy):

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \delta_t \quad (29)$$

Critic Update (Value Function):

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \beta \delta_t \phi(s_t) \quad (30)$$

where $\delta_t = r_{t+1} + \gamma V(s_{t+1}; \mathbf{w}) - V(s_t; \mathbf{w})$ is the TD error.

Advantage Actor-Critic (A2C)

Advantage Function:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (31)$$

A2C Update:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t \quad (32)$$

where $A_t = r_{t+1} + \gamma V(s_{t+1}; \mathbf{w}) - V(s_t; \mathbf{w})$ is the advantage estimate.

Benefits:

- Reduces variance compared to REINFORCE
- Faster convergence
- More stable training

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics**
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

Proximal Policy Optimization (PPO)

Objective: Prevent large policy updates that can destabilize training.

PPO-Clip Objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (33)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio.

Key Features:

- **Clipping:** Prevents large policy updates
- **Multiple Epochs:** Use data multiple times
- **Stable Training:** Works well in practice

Trust Region Policy Optimization (TRPO)

Trust Region Constraint:

$$\max_{\theta} \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t \right] \quad (34)$$

subject to $\mathbb{E}_t[KL(\pi_{\theta_{old}}(\cdot|s_t), \pi_{\theta}(\cdot|s_t))] \leq \delta$

Solution: Use conjugate gradient to solve constrained optimization.

Advantages:

- Theoretical guarantees
- Monotonic policy improvement
- Robust to hyperparameters

Soft Actor-Critic (SAC)

Maximum Entropy RL: Maximize both reward and entropy.

SAC Objective:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right] \quad (35)$$

where $\mathcal{H}(\pi(\cdot|s)) = -\sum_a \pi(a|s) \log \pi(a|s)$ is the entropy.

Benefits:

- **Exploration:** High entropy encourages exploration
- **Robustness:** Less sensitive to hyperparameters
- **Sample Efficiency:** Better than standard actor-critic

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning**
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

Multi-Agent MDP: Multiple agents interact in the same environment.

Challenges:

- **Non-stationarity:** Environment changes as other agents learn
- **Coordination:** Agents need to coordinate actions
- **Scalability:** Complexity grows with number of agents

Solution Approaches:

- **Independent Learning:** Each agent learns independently
- **Centralized Training, Decentralized Execution**
- **Communication:** Agents can share information

Centralized Training, Decentralized Execution:

Centralized Critic:

$$Q_i^\pi(o_1, \dots, o_n, a_1, \dots, a_n) \quad (36)$$

Decentralized Actor:

$$\pi_i(a_i | o_i) \quad (37)$$

Key Insight: During training, agents can observe other agents' observations and actions, but during execution, they only use their own observations.

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions**
- 11 Mathematical Foundations
- 12 Conclusion

Applications of Reinforcement Learning

Game Playing:

- AlphaGo, AlphaZero: Go, Chess, Shogi
- OpenAI Five: Dota 2
- StarCraft II: AlphaStar

Robotics:

- Manipulation tasks
- Locomotion
- Autonomous driving

Natural Language Processing:

- Dialogue systems
- Text generation
- Machine translation

Current Challenges and Future Directions

Current Challenges:

- **Sample Efficiency**: RL often requires many samples
- **Generalization**: Transfer learning across tasks
- **Safety**: Ensuring safe exploration and deployment
- **Interpretability**: Understanding learned policies

Future Directions:

- **Meta-Learning**: Learning to learn
- **Hierarchical RL**: Learning at multiple time scales
- **Causal RL**: Understanding cause and effect
- **Federated RL**: Learning across distributed systems

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations**
- 12 Conclusion

Convergence Analysis

Q-Learning Convergence:

Theorem

Q-learning converges to Q^ with probability 1 if:*

- ① *All state-action pairs are visited infinitely often*
- ② *Learning rates satisfy: $\sum_{t=0}^{\infty} \alpha_t = \infty$ and $\sum_{t=0}^{\infty} \alpha_t^2 < \infty$*

Policy Gradient Convergence:

Theorem

Under certain regularity conditions, policy gradient methods converge to a local optimum of the expected return.

Exploration vs Exploitation

Multi-Armed Bandit Problem:

- k arms, each with unknown reward distribution
- Goal: Maximize cumulative reward
- Trade-off: Explore new arms vs exploit known good arms

Upper Confidence Bound (UCB):

$$a_t = \arg \max_a \left[\hat{\mu}_a + c \sqrt{\frac{\log t}{N_a}} \right] \quad (38)$$

where $\hat{\mu}_a$ is the empirical mean and N_a is the number of times arm a was pulled.

Thompson Sampling:

- Maintain posterior distribution over arm parameters
- Sample from posterior and choose arm with highest sample
- Update posterior based on observed reward

Information Gain:

$$I(\theta; r_{t+1}|s_t, a_t) = \mathcal{H}(\theta|s_t, a_t) - \mathcal{H}(\theta|s_t, a_t, r_{t+1}) \quad (39)$$

Curiosity-Driven Exploration: Use prediction error as intrinsic reward.

Outline

- 1 Introduction to Reinforcement Learning
- 2 Bellman Equations
- 3 Dynamic Programming
- 4 Monte Carlo Methods
- 5 Temporal Difference Learning
- 6 Function Approximation
- 7 Policy Gradient Methods
- 8 Advanced Topics
- 9 Multi-Agent Reinforcement Learning
- 10 Applications and Future Directions
- 11 Mathematical Foundations
- 12 Conclusion

Key Takeaways

Fundamental Concepts:

- **MDP**: Mathematical framework for sequential decision making
- **Value Functions**: Quantify long-term value of states/actions
- **Bellman Equations**: Recursive relationships for value functions

Learning Algorithms:

- **Dynamic Programming**: When model is known
- **Monte Carlo**: Learn from complete episodes
- **Temporal Difference**: Learn from incomplete sequences
- **Policy Gradient**: Direct policy optimization

Modern Developments:

- **Deep RL**: Neural networks for function approximation
- **Multi-Agent RL**: Systems with multiple learning agents
- **Safe RL**: Ensuring safety during learning and deployment

Open Questions

If you are interested in these questions, please feel free to write an email to me (hpp@bimsa.cn) with your comments.

1. How can we design more sample-efficient RL algorithms?
2. What are the fundamental limits of exploration in RL?
3. How can we ensure safety and robustness in RL systems?
4. Can we develop theoretical frameworks for multi-agent RL?
5. How can RL be combined with other learning paradigms (supervised, unsupervised)?

Welcome inputs

Any comments?

Welcome your inputs!